Metamodeling for flooding alarm system by integrating functional inputs and outputs

Paul SLISSE[†] and Victor DAVODEAU[‡]

Project advisors: Pascal NOBLE [§], Olivier Roustant[¶], Rémy BARAILLE [‖]

Key words. metamodeling, flood, gaussian process, functionnal input/output, dimension reduction

[†]INSA Toulouse, 5^{th} year student,Department of applied mathematics, slisse@insa-toulouse.fr
[‡]INSA Toulouse, 5^{th} year student,Department of applied mathematics, davodeau@insa-toulouse.fr
[§]INSA Toulouse - Toulouse institute of mathematics, Toulouse, France, noble@insa-toulouse.fr
[¶]INSA Toulouse - Toulouse institute of mathematics, Toulouse, France, roustant@insa-toulouse.fr
[‖]SHOM - Toulouse institute of mathematics, Toulouse, France , remy.baraille@shom.fr

Table of Contents.

Introduction.

The French metropolitan littoral is 5500 linear kilometers long and up to 20000 linear kilometers if we consider overseas *départements* [2]. The littoral has also a high population density, 2.5 times higher than the rest of the country [3] with 975 littoral cities [2]. Even if climate change seems not to impact the frequency and power of the storms in France [6, 5], since 1980, more than 360 storms have affected the French metropolitan territory [5]. The need for accurate and rapid coastal flooding forecasts has always been important for the french administration. In France, coastal flooding warnings are part of the Vigilance Vagues Submersion (VVS) system, developed by Service Hydrographique et Océanographique de la Marine (SHOM) and operated by Météo-France. The current model used for the VVS system produces prediction at the scale on an entire *département* in France [4]. As a french *département* littoral is on average 240 kilometers long, the weather conditions can vary significantly and the flooding can be very different. Therefore, more research need to be conducted.

To tackle this lack of spatial information, the IMT (Insitut de Mathématiques de Toulouse) and SHOM are currently developing a high-performance numerical model called TOLOSA, which will be able to predict coastal flooding on a city-scale. Today's numerical models rely on the laws of physics to simulate marine hazards several days in advance. However, improving the accuracy of those models requires too much computation for real time decision making. For example, the current numerical model takes more than 10 minutes for 24 time steps. In time of crisis or for uncertainty studies, where we want to simulate thousands of different scenarios, with the current model, it is far too long.
To overcome this limitation, scientists have developed reduced models, also called metamodels. A metamodel can replace a parent model by learning from it to reproduce its outputs with a small amount of training data and then infer almost instantly [9].

Our objective is to develop a metamodel that will be the emulator of the TOLOSA framework. This model will learn from the existing simulator and be able to predict in real time on a city-scale. We will use Gaussian Process metamodels, one major advantage of which is their ability to output an uncertainty measure (Variance) [1]. We will develop this concept in section 1 more deeply. We will consider functional inputs and outputs, which are time dependent as both inputs (tide) and outputs are inherently time dependent. These are more complex than the classic scalar framework as now the inputs and/or outputs can be expressed as time series. Gaussian processes for a functional framework are relatively new and much less developed than the scalar framework [1]. Some articles are guiding us on the definition of the kernel for this new framework [1, 7].

This paper is organized as follows: We begin with a reminder of the principles of metamodeling, focusing in particular on Gaussian Processes and why they are well-suited to our spatial-temporal problem (section 1). We then detail the dimensionality reduction techniques we used to handle high-dimensional grid outputs: why it is necessary and how it can be achieved with FPCA (section 2). After that, we present the workflow of our metamodel, the different steps of the algorithm (section 3). Next, we examine the adaptations required to handle functional input data (section 4), followed by the modifications needed when the output itself varies over time (section 5). The framework is first validated on a synthetic benchmark, the Campbell function (section 6), before being applied to a real-world application in Saint-Malo (section 7).

1. Metamodeling with Gaussian Processes

1.1. What is metamodeling?

Let us take an example to explain briefly metamodeling: the modeling of car deformation after a crash. This example is highly inspired by the one presented by Olivier Roustant during his course [9].

We want to model the deformation of a future model of car on a wall to investigate the safety of this new model. We have some inputs (speed, angle) and a numerical model based on physical equations that can infer the deformation of the car after the crash.
But two problems arises :

- Time : Our model inference is really long. It takes several hours to run.
- Cost : The model inference is also computationally expensive, needing lots of energy and money to run

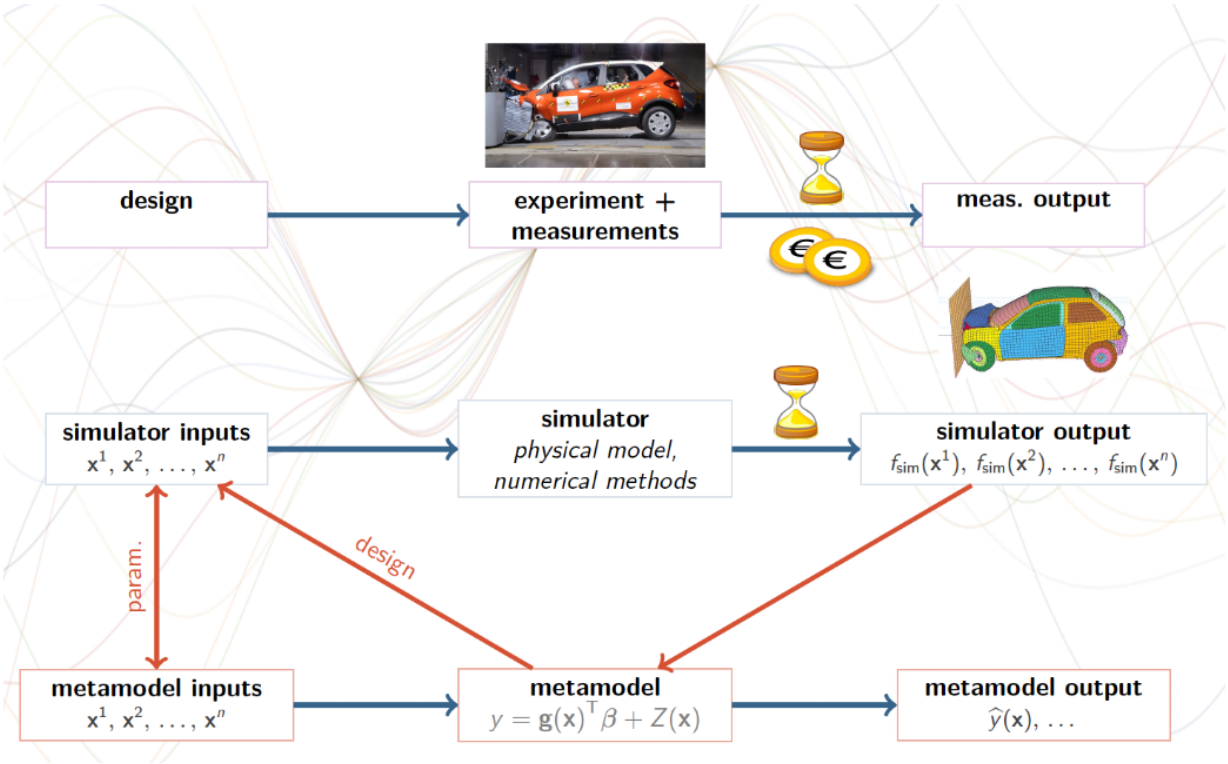

Figure 1: Example of metamodeling for car deformation presented in [**?**]

Here is when metamodeling comes. The goal of metamodeling is to create a "surrogate" model of an existing one but this surrogate model (also called metamodel) will be much faster, use a small amount of training data and can be used to choose new input points to test. A metamodel can be any statistical model : Linear Regression, Trees, Gaussian processes, ...

Let us come back on our example. We will create a metamodel to predict the deformation of the car given the same inputs as our initial model. We need first some realizations (we have to simulate some car crashes with different inputs) of our experiments as the metamodel is using the simulations of the physical model to be trained. After the training on those first training data, we use our metamodel to choose the new input points on which to launch the physical model. We launch the new experiments with our physical model (crash some other cars virtually) and give it to the metamodel to improve its accuracy. After that, our metamodel will be able to predict the car deformation with only the inputs (speed and angle) without the need of the expensive physical model (see example on Figure 1).

1.2. Metamodeling based on Gaussian processes

Let us delve further into the structure of a metamodel and examine its core components. We focus on the implementation of metamodeling using Gaussian Processes. Major advantages of Gaussian processes are that it can be built with only few datas and gives a measure of uncertainty in unvisited area.

Gaussian process is an interpolation method (the prediction is exact on the training points) that is based on similarity of the inputs to predict the output. Y is the output of the metamodel (the car deformation for example) and x, x' are model inputs (speed,angle vector for example). Y is defined to be a Gaussian process as following:

$$Y \sim \mathcal{GP}\left(m(x), k(x, x')\right)$$

m is the mean function and k is the covariance function also called the "kernel" function. The kernel k measure the similarity between two inputs x and x'. As a Gaussian process, for inputs $x_1, \ldots, x_n$, the random vector $(Y(x_1), \ldots, Y(x_n))$ is a Gaussian vector and its law is fully defined by the mean function m and kernel k [9].

The idea of Gaussian process regression also called kriging is to assume that the function that we want to reproduce (the initial model) is a sample path of some Gaussian Process (with certain mean and kernel). Our training data is considered as realizations of the Gaussian process at a certain input $(Y(x_i) = y_i)$ and we want to predict the realization at an unknown input. So we want to infer the law of Y conditioned on the observations $Y(x_i) = y_i, i = 1, \ldots, n$. Thanks to the properties of Gaussian vectors, this is still a GP (Gaussian Process). Given the vector of observed inputs $X = (x_1, \ldots, x_n)$ and a new unobserved input x we can infer the mean (1.1) and kernel (1.2) of the conditional process $Y(x)$ knowing $Y(x_i) = y_i, i = 1, \ldots, n$:

$$m_c(x) = m(x) + k(x, X)k(X, X)^{-1}(y - m(X)) \tag{1.1}$$

$$k_c(x, x') = k(x, x') - k(x, X)k(X, X)^{-1}k(X, x') \tag{1.2}$$

where $y = (y_1, \ldots, y_n)^\top$, $m(X) = (m(x_1), \ldots, m(x_n))^\top$, $k(x, X) = (k(x_1, x), \ldots, k(x_n, x))$, $k(X, x) = k(x, X)^\top$ and $k(X, X) = (k(x_i, x_j))_{1 \leq i,j \leq n}$.

The choice of the mean function m and the kernel k in (1.1) and (1.2) has a great impact on the properties of the associated Gaussian process. Hyperparameters tuning of the kernel is also a big part of GP regression and is done by Maximum Likelihood Estimation (MLE) but we will also not delve into it.

This GP will be our metamodel, the "reproduction" of the initial model. It's not really the reproduction, a sample path of the GP is an approximation of the initial model. If we have evaluation points $P = p_1, \ldots, p_n$, we can compute the means with m_c and covariance matrix with k_c of the Gaussian vector $Y(P)$ and a realization of this random variable will be a sample path of the GP, one of many approximations of the initial model. To get the prediction at an unobserved input x, we just need to evaluate the path sampled with at this input x. As we have the covariance/kernel of the GP, we can return an uncertainty measure that can be used to determine the input point that will improve the most the metamodel if added (example of EGO algorithm, see [9]). The fact that the prediction of the model is random allow us to draw multiple prediction that will be different to simulate different possible scenarios. This can also be used for sensitivity analysis but we didn't focus on that part during the project.

Even if it isn't the best way to improve the metamodel, we can use a predefined training design and not add any more data after a first training by using static space-filling designs such as Latin Hypercube [9].

1.3. Link with the project problem

Let us go back to the project problem. We have a numerical model that predict coastal flooding but too slow for real-time predictions and computationally expensive (more than 10 minutes for a simulation on 360 hearts of CPU). This also restricts the amount of data to train our model as we cannot use unlimited time and energy to run experiments on the numerical model. As we presented, metamodeling based on Gaussian processes is a relevant tool in this context. We will be able to choose the input design before running the experiments, the metamodel prediction will be accurate even with a small amount of training data and we will be able to get real-time prediction.

2. Dimension reduction for a 2D output: FPCA

Our outputs are flooding maps of high dimension that lives in $\mathbb{R}^{N\times N}$ où $N \in \mathbb{N}$. For example, in the toy model (that we will present in section 6), we work with $N = 64$ so the output maps are of dimension $\mathbb{R}^{64\times 64}$. Flattened, it becomes $\mathbb{R}^{4096}$. And it can be even higher depending on the resolution of the map (for real case study , see section 7). If we don't reduce the dimension and just flatten the map, two problems arises:

- Computation time : We will need to train N^2 GP, which will be too long as there is hyperparameter tuning for each GP
- Loss of information: There is obviously dependence between each pixel of the map, two neighbors pixels are highly correlated and by training one GP per pixel we will lose this spatial dependence.

We need to find a solution to reduce the dimension of our output but still conserving the spatial dependence. One efficient way to do this is by using Functional Principal Component Analysis (FPCA) [8].

While Principal Component Analysis (PCA) treats each observation as a vector, FPCA treats each observation as a continuous function $y_i(z)$ (z is the spatial space, here $\mathbb{R}^2$) belonging to a specific functional space $\mathcal{F}$. In this framework, the objective shifts from diagonalizing a covariance matrix to diagonalizing a covariance operator. Consequently, instead of searching for eigenvectors, FPCA seeks a set of eigenfunctions[8].

For us, a mathematical function is a clear object but it isn't for a computer that requires a discretized representation. Therefore, we must project our 2D maps onto a representation that preserves as much spatial information as possible before performing the analysis.

2.1. Projection onto a new functional basis

The first step of FPCA is to project the spatial maps $y_i(z), i = 1, \dots, n$ onto a functional basis $\Phi(z) = \{\phi_k\}_{k=1}^K$ of dimension $K \ll N$. This allows us to represent each map as a linear combination of basis functions:

$$y_i(z) = \sum_{k=1}^{K} \alpha_{i,k}\phi_k(z) \tag{2.1}$$

If we choose a basis designed for spatial data such as wavelets or B-splines,we ensure that the local and global dependencies of the maps are preserved in the coefficients $\alpha_{i,k}$. We have transformed our high-dimensional functional map into a lower-dimensional vector of coefficients $\alpha_i \in \mathbb{R}^K$.

2.2. Projection with PCA

By defining the vector $Y(z) = (y_1(z), \dots, y_n(z))^T$ of our training maps, it can be rewritten as a matrix product $Y(z) = A\Phi(z)$ where $A_{i,k} = \alpha_{i,k}$. It can be shown that looking for eigenfunctions of $Y(z)$ is equivalent to look for eigenvectors of A. In other words, FPCA on maps corresponds to performing PCA on the coefficients of the decomposition of maps in a functional space with a specific metric: the Gram matrix of the basis functions [8]. If the basis is orthogonal (as for wavelets), the Gram matrix is the identity matrix I and it simplifies to classic PCA.

Therefore, we will decompose our training maps in a functional basis and then perform centered PCA (center the data before PCA and add the mean of the data back when doing inverse PCA) on the coefficients of the decompositions of the training set. We transformed our training output space from $\mathbb{R}^{n\times N^2}$ to $\mathbb{R}^{n\times n_{pc}}$. This allow us to train a GP on each principal component with a very small computation time and a small loss of information.

We decided to explore two functional basis: wavelets and B-splines. To measure the gain of the decomposition on a new basis, we will also try to perform the PCA directly on the matrix of flattened maps of size $n \times N^2$.

B-spline decomposition B-spline bases are widely used for functional approximation and provide a smooth representation of functions or images using piecewise polynomials. In this approach, each spatial map $y_i(z)$ is expressed as a linear combination of B-spline basis functions:

$$y_i(z) = \sum_{k=1}^{K} c_{i,k} B_k(z),$$

where $B_k(z)$ denotes the k-th B-spline basis function and $c_{i,k}$ its associated coefficient for map i.

The B-spline basis is defined by a set of *knots* and a polynomial *degree*. The number of basis functions K depends on the number of knots and the chosen degree.

In practice, for a given training dataset, we first construct the B-spline design matrix B that evaluates all basis functions on the discretized spatial domain. Using least squares, we compute the coefficients C:

$$C = \arg\min_{C} \|BC^\top - Y^\top\|_F^2,$$

where Y is the matrix of flattened maps in the training set. Each row of C contains the coefficients for one training map.

Since the number of B-spline coefficients K can be large, a dimensionality reduction step is required. To this end, we apply principal component analysis (PCA) to the matrix of B-spline coefficients.

$C \in \mathbb{R}^{n_{\text{train}} \times K}$, where each row corresponds to one training output. The coefficients are first centered by subtracting their empirical mean:

$$\bar{C} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} C_i, \qquad \tilde{C}_i = C_i - \bar{C}.$$

PCA is then performed on the centered matrix $\tilde{C}$. As the B-splines basis functions are not orthogonal, it is not equivalent to FPCA. The recovery between two B-splines basis functions is null for almost every couple and is really small between neighbors encouraging us to still perform classic PCA. This yields a set of orthonormal principal directions $\{v_k\}_{k=1}^{K}$ ordered according to the variance they explain. Only the first n_{pc} components are retained, providing a low-dimensional representation of the coefficients:

$$z_{i,k} = \langle \tilde{C}_i, v_k \rangle, \qquad k = 1, \ldots, n_{\text{pc}}.$$

As a result, each coefficient vector can be approximated in the reduced space as

$$C_i \approx \bar{C} + \sum_{k=1}^{n_{\text{pc}}} z_{i,k}\, v_k.$$

This procedure combines the smooth approximation properties of B-splines with dimensionality reduction via PCA, enabling efficient Gaussian process modeling while preserving the main spatial structures of the outputs. The overall procedure is summarized in Algorithm 2.1.

Wavelet decomposition Wavelet basis is a well-known basis in signal and image theory for decomposition of signals or images considered as functions. In this basis our maps rewrite:

$$y_i(z) = \sum_{k=1}^{K} \alpha_{i,k} \psi_k(z) = A_i \Psi^\top(z)$$

where $\psi_k(z)$ denotes the k-th wavelet basis function, $\alpha_{i,k}$ its associated coefficient for map i and A is the matrix of the α's: $A_{i,k} = \alpha_{i,k}$

As wavelet decomposition is returning a vector of the same size as the discretization, we want to

Algorithm 2.1 Dimension reduction using B-spline decomposition

Require: Training outputs y_{train}, knot vectors t, spatial grid z, number of principal components n_{pc}, polynomial degree d

1: Construct the B-spline basis matrix B by evaluating the B-spline basis functions on the spatial grid z

2: Compute the B-spline coefficient matrix $C \in \mathbb{R}^{n_{\text{train}} \times K}$ by least squares:

$$C = \arg\min_{C} \|BC^\top - Y^\top\|_F^2$$

3: Compute the empirical mean of the coefficients:

$$\bar{C} = \frac{1}{n_{\text{train}}} \sum_{i=1}^{n_{\text{train}}} C_i$$

and center the coefficients: $\tilde{C}_i = C_i - \bar{C}$

4: $y_{train-PCA} \leftarrow$ Perform PCA on the centered coefficient matrix $\tilde{C}$ and retain the first n_{pc} principal components.

5: **return** PCA projected outputs $y_{train-PCA}$, PCA projection matrix $V_{\text{B-splines}}$, residual mean $\bar{C}$, B-spline basis matrix B

reduce the number of coefficients that we keep. We could choose to keep only the approximation coefficients (whose number decrease with the resolution J of the basis) but we decided to use another method presented in [8] using energy.

As the wavelet basis is orthonormal, one can write a decomposition of the spatial energy: $\|y_x\|_2^2 = \sum_{k=1}^{K} \alpha_k(x)^2$ (here we replaced i by x as we define the equality for every possible input x and not only for the training set maps). Each coefficient α_k corresponds to a part of this energy and we can quantify the importance of each coefficient in this energy with $\frac{\alpha_k(x)^2}{\sum_{k'=1}^{K} \alpha_{k'}(x)^2}$. As this ratio depends on the input x, it is not accurate to suppose that the important coefficient are the same for every input. Thus, we will use the mean energy ratio :

$$\lambda_k = \mathbb{E}\left[\frac{\alpha_k(x)^2}{\sum_{k'=1}^{K} \alpha_{k'}(x)^2}\right]$$

that we will approximate by the empirical mean on the training set.
After we get those λ_k coefficients, we will chose the $\tilde{K} < K$ wavelet basis functions to keep by ordering the λ_k and using a threshold $p \in [0, 1]$ such that:

$$\sum_{k=1}^{\tilde{K}+1} \lambda_{(k)} > p \tag{2.2}$$

We sort accordingly the α's and the ψ's to the λ's and we can rewrite the decomposition in the wavelet basis, for the training set:

$$y_i(z) = \sum_{k=1}^{\tilde{K}} \alpha_{(k)}(x_i)\psi_{(k)}(z) \; + \sum_{k=\tilde{K}+1}^{K} \alpha_{(k)}(x_i)\psi_{(k)}(z), \quad \text{with } \alpha_{i,(k)} = \alpha_{(k)}(x_i)$$

With the vectorized notation: $Y(z) = \tilde{A}\tilde{\Psi}^\top(z) + \bar{A}\bar{\Psi}^\top(z)$. We will perform the PCA on the matrix $\tilde{A}$ to get the final transformation of the training outputs. The other wavelet coefficients are predicted by empirical mean $\bar{\alpha} = \frac{1}{n}\sum_{i=1}^{n} \bar{A}_{i,k}$. The overall procedure is summarized in Algorithm 2.2.

Algorithm 2.2 Dimension reduction using wavelet decomposition

Require: x_{train}, y_{train}, number of principal components n_{pc}, threshold p, resolution J

1: Compute wavelet decomposition of each output map of the training set to get the matrix A

2: Compute λ_k for each function of the wavelet basis:

$$\lambda_k = \mathbb{E}\left[\frac{\alpha_k(x)^2}{\sum_{k'=1}^{K}\alpha_{k'}^2(x)}\right] = \frac{1}{n_{train}}\sum_{i=1}^{n_{train}}\frac{A_{i,k}^2}{\sum_{k'=1}^{K}A_{i,k'}^2}$$

3: Order the λ_k coefficients and retain $\tilde{K}$ so that (2.2) is satisfied

4: Separate the $\lambda_{(k)}$ in two sets depending if $k < \tilde{K}$ or not

5: Separate the wavelet basis functions $\psi_k$into two sets depending of the set of their associated λ_k

6: Separate the matrix of the wavelet decomposition of the training set int two matrices $\tilde{A}$ and $\bar{A}$ depending of the set of the basis functions

7: Compute the empirical mean of the wavelet coefficients:

$$\bar{\tilde{A}} = \frac{1}{n_{\text{train}}}\sum_{i=1}^{n_{\text{train}}}\tilde{A}_i$$

$$\bar{\alpha} = \frac{1}{n_{\text{train}}}\sum_{i=1}^{n_{\text{train}}}\bar{A}_i$$

and center the coefficients: $\tilde{A}_i^{PCA} = \tilde{A}_i - \bar{\tilde{A}}$

8: $y_{train-PCA} \leftarrow$ Perform PCA on the centered coefficient matrix $\tilde{A}^{PCA}$ and retain the first n_{pc} principal components.

9: **return** PCA transformed y_{train} : $y_{train-PCA}$, PCA components :$V_{wavelet}$,residual mean: $\bar{\tilde{A}}$, empirical mean :$\bar{\alpha}$, wavelet basis :Ψ

We here only presented the *encoding* part, we will present the *decoding* part in section 3. We tested the encoding-decoding to be sure of the capacity of reconstruction of the method to be sure that our FPCA projection is sufficient to recompose our maps. Result of this reconstruction error are given in section 6 for the toy model and section 7 for the real-case study.

3. Workflow of our metamodel

Now that we have reduced our training maps, we can perform the Gaussian process part. Here we will synthesize the whole workflow of our model. We present the classical approach where our inputs are scalars and our outputs are scalar maps. We will discuss in the next parts what we need to change if we are not in the scalar framework but in the functional one.

After reducing the dimension of the training maps, we give it additionally to the training input set to our Gaussian process regression (GPR) function. As we mentionned in section 1, we need to choose a kernel for our Gaussian processes. The choice of the kernel is a highly important part of Gaussian processes but we didn't focus too much on the best choice of kernel and choose a classic one, the squared exponential (SE) kernel:

$$k(x, x') = \sigma^2 \exp\left(-\frac{\|x - x'\|_2^2}{2\ell^2}\right) \tag{3.1}$$

where x and x' are two inputs $\in \mathbb{R}^d$, $\|.\|_2$ is the Euclidean norm on $\mathbb{R}^d$, σ and ℓ are hyper-parameters of the kernel (respectively the variance and the lengthscale) that will be tuned during the training phase by MLE. This multidimensional SE kernel on $\mathbb{R}^d$ is built by tensor product of d 1-dimensional SE kernel defined by the difference of the input squared instead of the norm.

We then fit one Gaussian process per principal component of the training maps.

Once hyper-parameters have been tuned, we want to predict flooding maps with new inputs. Given a new test input x^*, the Gaussian processes predict the scores associated with the principal components. This can be achieved with randomness as each Gaussian process outputs a mean value m and a variance value v^2. We can then sample from a $\mathcal{N}(m, v^2)$ to get the prediction of the Gaussian process. This way for the same input x^* we can have different outputs and simulate different possible scenarios. We decided not to use the variance part and only use the mean as our prediction. So for our implementation, the same input x^* will always give the same prediction. As the mean is the naive choice that will emprirically minimize the error with the real value , it is a strong decision and potentially not the more accurate for this project. We will discuss further this point in a discussion in section 8.

After we get the prediction of the principal components, we need to reconstruct the flooding map by doing the inverse transformation than in section 5. We denote $\widehat{z}(x^*) \in \mathbb{R}^{n_{\text{pc}}}$ the vector of predicted PCA scores, we can recover the vector of the decomposition in the basis :

- For wavelet :
$$\widehat{\alpha}(x^*) = concatenate\left(\widehat{z}(x^*)V_{wavelet}^\top, \bar{\alpha}\right) + \bar{\bar{A}}$$
- For B-splines :
$$\widehat{c}(x^*) = \widehat{z}(x^*)V_{B-splines}^\top + \bar{C}$$

where the V's are the PCA decomposition matrices (Algorithm 2.2 and Algorithm 2.1).

Now that we have the prediction in the functional basis, we just need to project it back to the functional space of the outputs by performing inverse transformation:

- For wavelet:
$$\widehat{y}(z; x^*) = \sum_{k=1}^{K} \widehat{\alpha}_k(x^*)\psi_k(z),$$
- For B-splines:
$$\widehat{y}(z; x^*) = \sum_{k=1}^{K} \widehat{c}_k(x^*)B_k(z),$$

For the method using only PCA (no projection on a functional basis), there is just one step for the reconstruction :

$$\widehat{y}(z; x^*) = \widehat{z}(x^*) V_{PCA}^\top$$

This reconstruction procedure enables efficient prediction of high-dimensional. The overall computational cost remains low, making the approach well suited for real-time or near real-time surrogate modeling.

We summed-up the training phase in Algorithm 3.1 and the prediction phase in Algorithm 3.2.

Algorithm 3.1 Training phase of the Gaussian process metamodel

Require: x_{train}, y_{train}, number of principal components n_{pc}, kernel : $\mathcal{K}$, parameters θ, (knots t, domain grid z, polynomial degree d for B-splines) , (threshold p, resolution J for Wavelet)
Ensure: $\mathcal{M}$ (list of optimized GP models)

Apply Algorithm 2.2 for wavelet decomposition and PCA, Algorithm 2.1 for B-Splines decomposition and PCA or perform directly PCA on y_{train} if you don't want to use a functional decomposition
Flatten functions into vectors :$X_{flat} \leftarrow \text{reshape}(X_{train}, [N_{train},\ height * width\ ])$
$\mathcal{M} \leftarrow \emptyset$
Instantiate the kernel : $k = \mathcal{K}(\theta)$
for $i = 1$ **to** n_{pc} **do**
    Select the i-th principal component score: $y^{(i)} \leftarrow y_{train}[:, i]$
    Define GP model : $m_i \leftarrow \text{GPR}(data = (X_{flat}, y^{(i)}), \text{kernel} = k, \text{mean} = \text{constant})$
    Hyperparameter tuning: minimize $\mathcal{L}(m_i)$ w.r.t. trainable parameters θ
    $\mathcal{M} \leftarrow \mathcal{M} \cup \{m_i\}$
end for
return $\mathcal{M}$

Algorithm 3.2 Prediction of the Gaussian Process metamodel

Require: list of n_{pc} trained models : $\mathcal{M}$, new input :x_{new}, (PCA components : V_{PCA} for PCA only) , (PCA components : $V_{B-spline}$, Residual mean : $\bar{C}$, B-spline basis: B for B-spline), (PCA components : $V_{wavelet}$, Residual mean: $\bar{\bar{A}}$ empirical mean: $\bar{\alpha}$, wavelet basis: Ψ for wavelet)
Ensure: Predicted flooding map $\hat{y}$

$Means \leftarrow \emptyset$
for $i = 1$ **to** n_{pc} **do**
    Predict latent mean for the i-th principal component $\mu_i \leftarrow \mathcal{M}_i.\text{predict}(x_{new})$
    $Means \leftarrow Means \cup \{\mu_i\}$
end for
Perform inverse PCA : $y_{PCA} \leftarrow MeansV^\top$ (where V is the PCA components matrix of the chosen method: PCA only, B-spline or wavelet)
{Perform inverse projection of the functional decomposition}
if Wavelet **then**
    $\hat{y} \leftarrow \left(concatenate(y_{PCA} + \bar{\bar{A}}, \bar{\alpha})\right) \Psi^\top$
else if B-spline **then**
    $\hat{y} \leftarrow (y_{PCA} + \bar{C})B^\top$
else
    $\hat{y} \leftarrow y_{PCA}$
end if
return $\hat{y}$

4. Kernel changes for functional inputs

The goal of the project is to be able to predict coastal flooding on a certain time window. As the weather conditions (the inputs) are varying through time, we need to adapt our method to be able to handle functional inputs: each weather condition is not a scalar anymore but a scalar function of the time t.

In the classical Gaussian process framework (as we presented in last two parts) inputs are assumed to lie in a finite-dimensional Euclidean space ($\mathbb{R}^d$, and similarity between inputs is typically measured using a standard norm such as the Euclidean distance. This assumption is no longer valid when the inputs are functions defined over a time interval. In such a setting, representing each input as a high-dimensional vector obtained by time discretization would lead to a loss of structural information and potentially poor performance, due to the curse of dimensionality and the lack of temporal coherence.

To properly account for temporally dependent inputs, the Gaussian process kernel must therefore be adapted to operate directly on functional objects. In a general case, each input is composed of d functions defined on a common time interval $[0, T]$. A natural way to compare such inputs is to measure their discrepancy in an integral sense, rather than pointwise. This motivates the use of a distance based on the L^2 norm between functions.

Let $X = (f_1, \dots, f_d)$ and $X' = (g_1, \dots, g_d)$ be two functional inputs, where each component f_i and g_i belongs to $L^2([0, T])$. A distance based solely on the L^2 norm of the functions captures discrepancies in amplitude but does not fully account for differences in temporal behavior. Two signals may be close in value while exhibiting very different oscillatory patterns or smoothness properties. To address this limitation, we enrich the distance definition by incorporating information on the temporal derivatives of the functions. We define the Sobolev norm of our input space $\mathbb{X} = (L^2([0, T]))^d$:

$$\|X - X'\|_{H^1}^2 = \sum_{i=1}^{d} \|f_i - g_i\|_{L^2([0,T])}^2 + \alpha \sum_{i=1}^{d} \left\|f_i' - g_i'\right\|_{L^2([0,T])}^2, \tag{4.1}$$

where $\alpha > 0$ is a weighting parameter that controls the relative influence of temporal smoothness in the kernel. The first term measures pointwise discrepancies between the functions, while the second penalizes differences in their temporal variations.

As $\mathbb{X}$ is a Hilbert space, we can use a radial kernel with the norm defined previously in (4.1) to define our new kernel [9]. To stay coherent with the previous kernel, we can still use the square exponential kernel (3.1), as it is a radial kernel. The σ^2 and $1/2\ell^2$ are added by stability of kernels by multiplication of kernels and multiplication by a constant [9]. The Gaussian process kernel is then defined as

$$k(X, X') = \sigma^2 \exp\left(-\frac{1}{2\ell^2} \|X - X'\|_{H^1}^2\right), \tag{4.2}$$

We thus have 3 hyperparameters for this kernel : (σ, ℓ, α).

In practice, the functions are observed on a discrete time grid, and both integrals are approximated using numerical quadrature. The derivative term is computed using finite differences. This formulation allows the kernel to distinguish between functional inputs that may be close in average value but differ significantly in their temporal dynamics.

By embedding this functional distance into a squared exponential kernel, we preserve the desirable properties of Gaussian processes—smoothness, flexibility, and interpretability—while extending their applicability to temporally structured functional inputs. This kernel modification is a key component that enables the proposed methodology to handle functional inputs in a principled and efficient manner.

For the numerical implementation, to cope with the large size of the kernel matrix induced by functional inputs, the Gram matrix is computed in a blockwise fashion, which significantly reduces memory requirements and enables efficient training of the model.

5. Adaptation for 2D functional outputs

We are now able to handle functional inputs, the final step to be able to predict coastal flooding on a time window is to provide a functional output and not a 2D map anymore. Our output is now a 3D function, maps that are varying through time.

As the inputs are unchanged, we have nothing to change in the Gaussian process part. But as we change the shape of the output, we have to adapt the dimension reduction part that we have done in section 5. 3 choices are now possible:

- Naive way: Flatten the 3D array of time dependent maps to get a vector and directly reuse the framework as in section 5
- 3D decomposition: As we decomposed in a 2D functional basis in section 5, we can decomposed in a 3D functional basis (3D B-splines or 3D Wavelets)
- Research way: Try a time dimension reduction before applying the 2D dimension reduction as in section 5 or find in the litterature a way to handle this 3D data

We decided to use the naive way for PCA only and also for wavelet (as our implementation of 2D Wavelet gave unsucessfull results, same as 1D Wavelets). For B-splines, we tried a 3D decomposition but our implementation didn't gave the expected results (see section 6).

So no major changes for handling functional output in our case. This is a first approach and obviously not the most optimized one for our problem. This is the part where the most improvement can be done on our method. By flattening, we lose a lot of temporal dependence and we are quite sure that better methods exists to handle such 3D data to conserve in the same time spatial and temporal dependencies. We did not spend too much time on it to be able to test our model both on the toy example and real case study.

6. Application to a toy example

Before applying our methodology to a real-world case, we start by considering a toy example. This allows us to simplify the problem by avoiding temporal constraints. Indeed, as mentioned earlier, the function (=physical model) we aim to estimate is computationally expensive to evaluate.

We therefore require a simple function that takes scalar inputs and returns a scalar output for each spatial position (z_1, z_2) in a subdomain of $\mathbb{R}^2$.

Consequently, we work with the Campbell2D toy function.

6.1. The toy simulator: Campbell function

The Campbell2D function is well suited to our purpose. It takes eight scalar inputs and produces a spatial map as output :

$$\begin{aligned} f : [-1,5]^8 &\longrightarrow L^2\big([-90,90]^2\big), \\ x = (x_1,\dots,x_8) &\longmapsto y_x(z), \end{aligned}$$

With

$$\begin{aligned} y_x(z_1,z_2) = x_1 \exp\left(-\frac{(0.8z_1+0.2z_2-10x_2)^2}{60x_1^2}\right) \\ + (x_2+x_4)\exp\left(\frac{(0.5z_1+0.5z_2)x_1}{500}\right) \\ + x_5(x_3-2)\exp\left(-\frac{(0.4z_1+0.6z_2-20x_6)^2}{40x_5^2}\right) \\ + (x_6+x_8)\exp\left(\frac{(0.3z_1+0.7z_2)x_7}{250}\right). \end{aligned}$$

For our numerical experiments, the spatial domain must be discretized. We choose to work with a uniform grid of size 64×64.

6.2. The classical approach: Scalar to scalar scenario

Drawing inspiration from the thesis [8], we have first considered the case where the inputs are scalar quantities (such as average wave height, daily temperature, etc) and the output is also scalar (for instance, the mean water height over a day). Figure 2 illustrates examples of outputs of the Campbell2D function for three different input configurations.

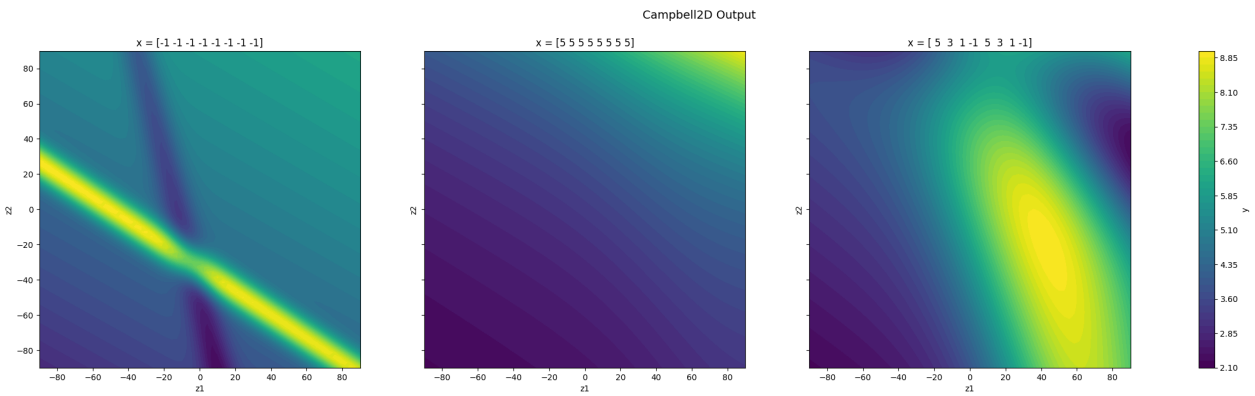

Figure 2: Example of Campbell2D outputs

We then constructed a training dataset and a test dataset. The training set contains 200 samples, while the test set consists of 1000 samples. In order to obtain a high-quality design of experiments, we adopted an optimized Latin Hypercube Sampling strategy. This approach ensures

a good space-filling property and a uniform marginal distribution for each input variable. Figure 3 illustrates the design of experiments. The test data are generated by independently sampling each input variable from a uniform distribution over the interval $[-1, 5]$.

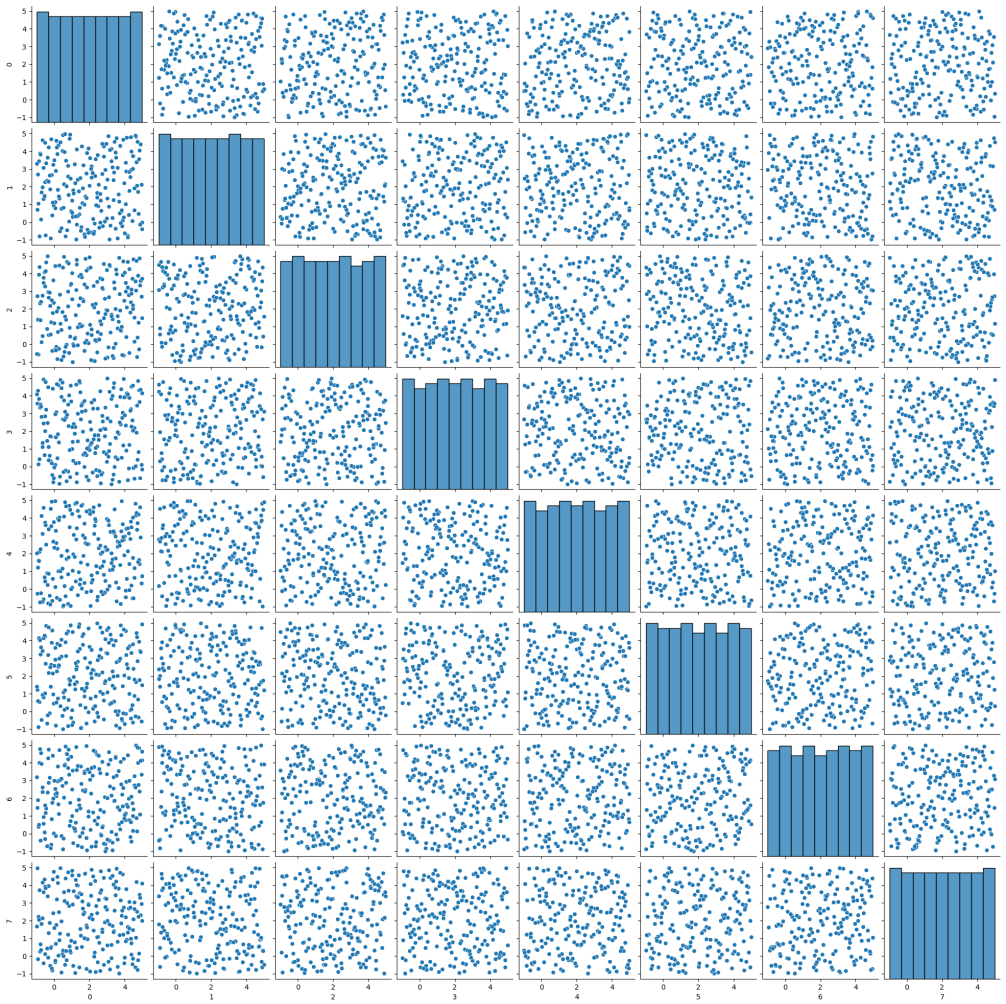

Figure 3: Design of experiment of the 8 scalar input. We observe the distribution of each x_i in the diagonal.

Since the simulator function (Campbell2D) is fully known, all output maps can be computed in advance and stored in memory for both the training and test datasets. To assess the robustness of the proposed methodology, we add a small amount of white noise to the input variables. Indeed, under real-world conditions, input values are never exact, as they typically originate from sensors or meteorological data.

We can now implement our methodology to train the Gaussian process models. Three approaches are investigated: PCA only, FPCA using B-spline bases, and FPCA using wavelet bases. For the B-spline–based FPCA approach, we used the following knot vector:

$$[-90, -90, -80, -70, -60, -50, -40, -30, -20, -10, 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 90, 90]^2$$

Based on the recommendations provided in [8], we retain only five principal components in the PCA step. A separate Gaussian process model is then trained for each component (section 3).We select the squared exponential kernel, defined in (3.1).

After hyperparameter optimization, we obtain the values in Table 1 for the first principal component of each method.

hyperparameter	PCA	FPCA using B-splines	FPCA using wavelets
$m(x)$	150.6	69.7	150.7
ℓ	13.3	14.1	13.3
σ^2	695.0	269.0	695.0

Table 1: Gaussian process hyperparameters for the first principal component using the three considered methods.

Once the Gaussian process hyperparameters have been identified, new outputs can be predicted efficiently. An example is shown below in Figure 4.

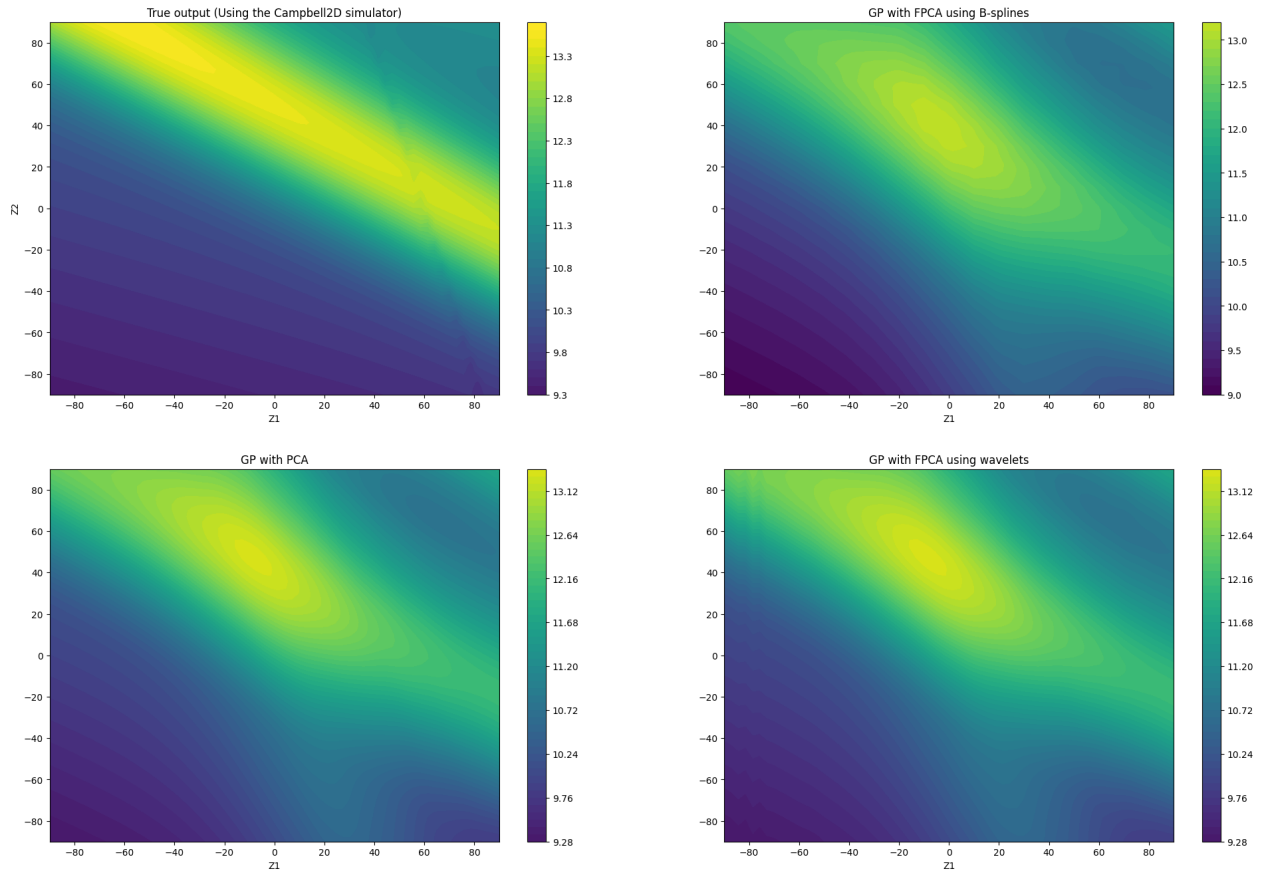

Figure 4: Example of reconstruction of an output

On each graph, we observe the water level over our domain $[-90, 90]^2$. The more yellow the color, the higher the water level, while the bluer the color, the lower the water level. The top-left graph represents the true output, while the other three graphs show our reconstructions.

To evaluate the performance of the proposed methods, we compute the Root Mean Squared Error (RMSE) for each pixel j :

$$\text{RMSE}_j = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_{i,j} - \hat{y}_{i,j})^2} \tag{6.1}$$

The RMSE measures the average magnitude of the errors between the predicted values $(\hat{y}_{i,j})$ and the true observed values $(y_{i,j})$ for each pixel j across all N samples. It provides a quantitative assessment of the accuracy of the reconstruction or prediction: the lower the RMSE, the better the model's performance in capturing the true water height distribution. The RMSE is in the same units

as the original data (water height), making it intuitive to interpret and compare across different methods or configurations. This allows us to generate spatial RMSE maps Figure 5.

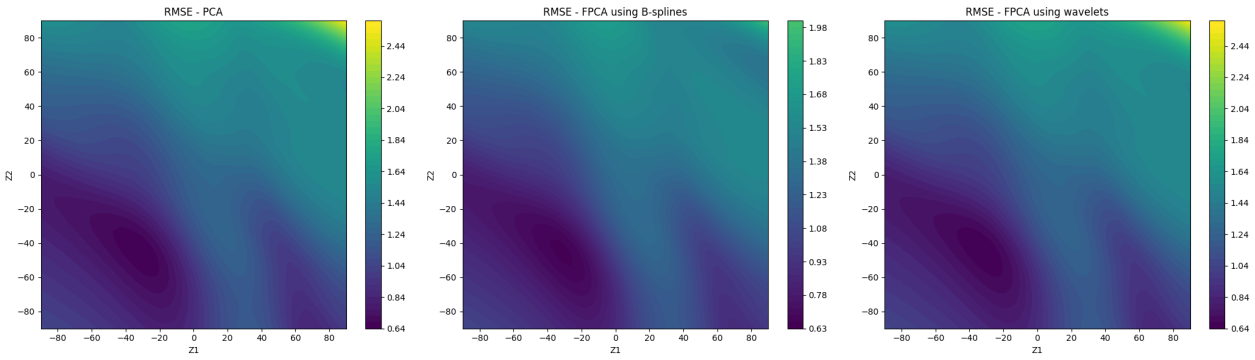

Figure 5: RMSE maps

As we said at the end of section 5, our dimension reduction create a reconstruction error. To estimate which amount of error comes with reconstruction or prediction, we are going to predict values on the train set. As GPR, is an interpolation method, the prediction is exact and the error on the training data thus only depend on the reconstruction error. We got the following results:

	PCA	FPCA using B-splines	FPCA using wavelets
RMSE	0.725	0.764	0.724

Table 2: Estimation of the reconstruction error for Scalar Campbell

As we compare the reconstruction error and the global RMSE error, very satisfactory results are obtained for all three methods. We observe maximum 10% global error in average for a map and a significant amount is coming from the reconstruction. B-splines seems to perform a little better than the two other methods on the right hand corner. In particular, we are able to reproduce the results previously reported in [8] where they work with the same toy function and FPCA. This validates the methodology for scalar input and output. Let us now extend the method to more general data and output.

6.3. Adding temporality to the input

We now consider the case where the inputs exhibit temporal dependence. For instance, we aim to use weather condition sampled every 10 minutes over a 6-hour period. To this end, we consider a vector of 8 functions (as Campbell function has 8 inputs) $X = (f_1, f_2, \dots, f_8)$, where each function f_i is defined on the time interval $[0, T]$ and takes values in $[-1, 5]$. These functions model periodic and random signals, constructed using a truncated Fourier decomposition.

For each parameter $\alpha_i \in [0, 2],\ i = 1, \dots, 8$, we generate a function f_i from a complex Fourier series of the form

$$f_i(t) = 2 + A \sum_{k=0}^{M-1} \frac{a_{i,k}}{(1+k^2)^{\alpha_i}}\ e^{i\omega_i k t}, \qquad t \in [0, T].$$

In this expression:

- M is the number of retained Fourier modes;
- the random coefficients $a_{i,k}$ are drawn uniformly from $[-1, 1]$;
- the term $1/(1+k^2)^{\alpha_i}$ enforces a controlled decay of high-frequency amplitudes;
- ω_i is a random angular frequency, defined as $\omega_i = \frac{2\pi}{U_i}$, where U_i is uniformly sampled in the interval $[0, T]$;
- $A = 2$ is a global amplitude used to center the signals within the interval $[-1, 5]$ after adding a constant bias;

- finally, only the real part of the series is retained in order to obtain a real-valued signal.

Thus, for each value of α_i, we obtain a periodic function whose regularity depends directly on the parameter α_i: the larger α_i is, the faster the decay of the Fourier coefficients, and the smoother the resulting function.

An example of such generated signals is shown below in Figure 6. We chose to use 8 signals to

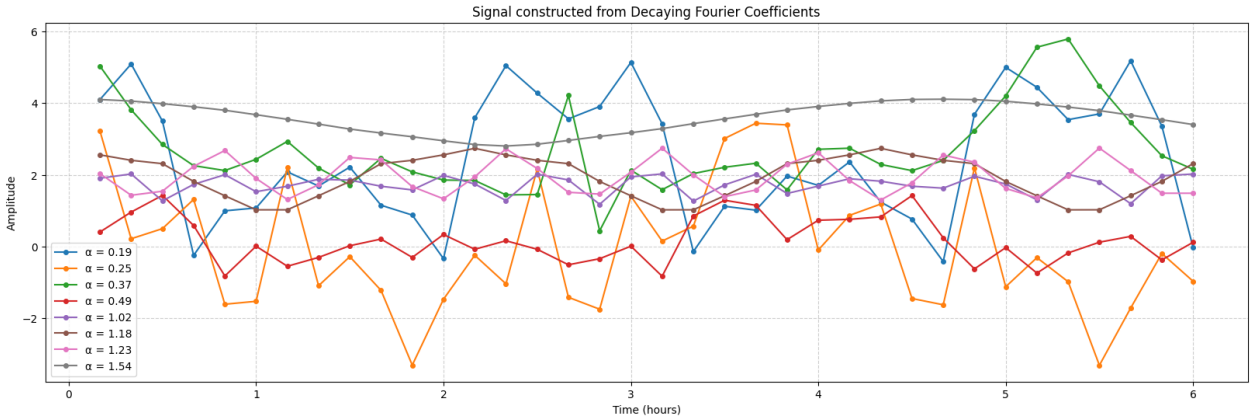

Figure 6: Eight signals, each consisting of 36 samples (one every 10 minutes)

reuse the Campbell2D function and stay as close to reality as possible. We selected a 6-hour time window, which roughly corresponds to a tidal cycle. Measurements were taken every 10 minutes, as this aligns with the approximate data frequency we expect to work with in the future.

We observe a wide diversity of signals, exhibiting varying degrees of smoothness and periodicity.

We then considered how to construct our design of experiments. To this end, we adopted a *maximin criterion*: we generated a large number of candidate designs (1000). Each design consists of 200 points. Recall that a point corresponds to a collection of 8 functions defined on the interval $[0, T]$.

For each design, we computed the pairwise distances between all points. This required defining a distance between such functional objects. We chose an L^2-type distance, defined as follows.

Let $X = (f_1, \ldots, f_8)$ and $X' = (g_1, \ldots, g_8)$ be two points of the design, where each component is a function on $[0, T]$. The distance between X and X' is given by

$$d(X, X') = \left(\sum_{i=1}^{8} \|f_i - g_i\|^2_{L^2([0,T])} \right)^{1/2},$$

where

$$\|f_i - g_i\|_{L^2([0,T])} = \left(\int_0^T (f_i(t) - g_i(t))^2 \, \mathrm{d}t \right)^{1/2}.$$

Finally, among all generated designs, we retained the one that maximizes the minimum pairwise distance, in accordance with the maximin criterion.

Figure 7 provides an intuitive illustration of this procedure.

Computing this design of experiments requires a significant computational effort. For this reason, the raw generated values were stored in a Git repository (see section 8). In addition, a test dataset consisting of 2000 samples was generated and stored in the same repository.

Now that the design of experiments has been constructed, we must define the type of output required. To this end, we again rely on the Campbell function. Recall that the inputs consist of eight functions, each sampled at 36 time steps. To construct the outputs, the Campbell function is evaluated at each of these time steps, after which three different post-processing operations are applied. Applying the Campbell function at each time step can thus be interpreted as producing

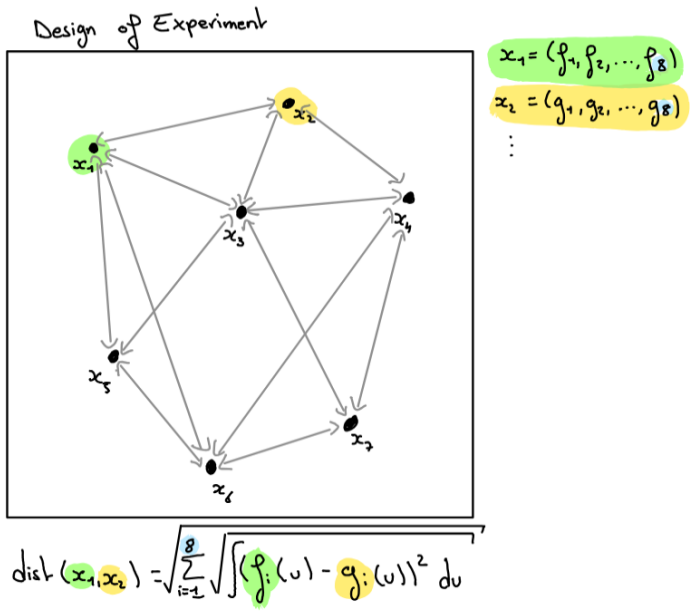

Figure 7: Schematic representation of pairwise distances in the design of experiments.

a water height map at each instant in time. For each spatial pixel, we then retain three summary statistics over the time interval: the mean, minimum, and maximum water height. This procedure yields an output that no longer depends explicitly on time. Figure 8 illustrates the outputs. This approach effectively addresses the question of whether an area will be flooded, provides an estimate of the proportion of water that may overflow, and potentially indicates if a structure will be overtopped.

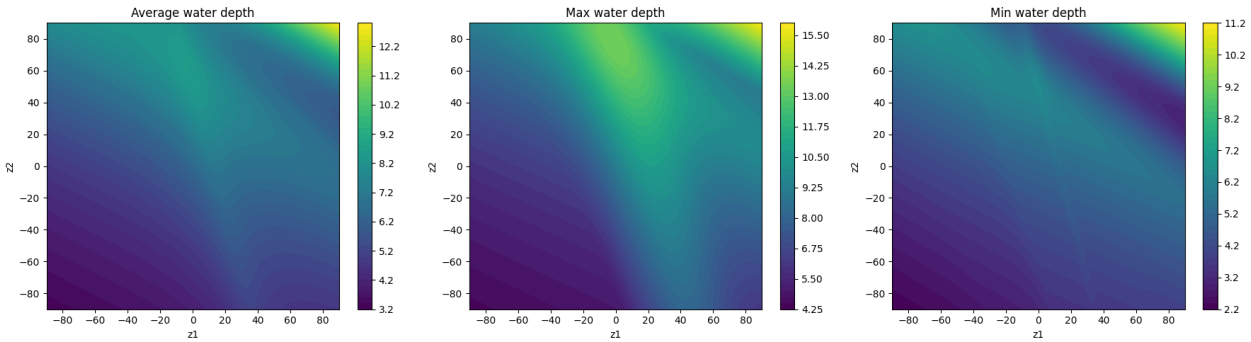

Figure 8: Example of output obtained from a functional input.

For the remainder of this section, we focus on the minimum water height. At this stage, all necessary components are available to construct the training and test datasets. To handle time dependent functional inputs, the Gaussian process framework must be adapted to operate on functions rather than finite-dimensional vectors. As detailed in section 4, we rely on a kernel specifically designed for functional inputs (4.2), based on a distance defined on the functional space (4.1). The kernel hyperparameters (σ^2, ℓ, α) are learned by maximizing the marginal log-likelihood.

Now that the Gaussian process models have been adapted to functional inputs, the remainder of the methodology remains identical to that presented previously, since the outputs are objects of the same nature. We can therefore directly examine the RMSE maps obtained for the three methods and reconstruction error Table 3 , as shown in Figure 9.

We find similar results as for the fully scalar framework. The absolute error is sufficiently small (maximum 10% error in average with significant part of reconstruction error). PCA only and

	PCA	FPCA using B-splines	FPCA using wavelets
RMSE	0.568	0.648	0.569

Table 3: Estimation of the reconstruction error for functional input Campbell

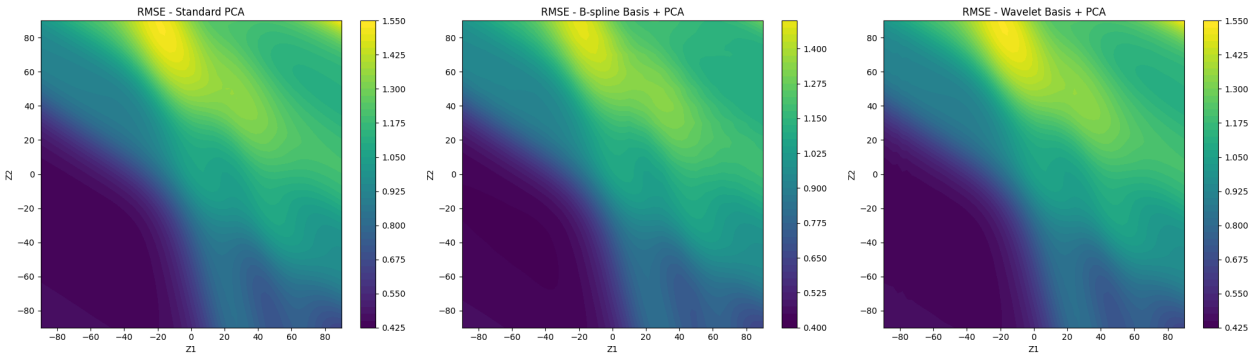

Figure 9: RMSE maps for functional inputs

Wavelet have similar performance while B-spline outperform a little bit the two other method on the right hand corner of the map leading to more smooth maps.

6.4. Adding temporality to the output

Now that we are able to handle functional inputs, we extend the methodology to time dependent output. We consider keeping the same inputs as in the previous section, but this time the output is a water height map that evolves over time.

To simulate this setting, we once again use the Campbell2D function. Recall that the inputs consist of a set of 8 functions evaluated at 36 time steps. To construct the output, we simply evaluate the function at these 36 time steps, yielding an object of dimension $64 \times 64 \times 36$, which represents the output. The outputs can be visualized using animated GIFs, which are available in the Git repository (see section 8, go to Cas_jouet_Campbell/output-GIF/maps or maps2 and watch the hauteur_eau_réelle).

For the remainder of the study, we can apply our methods directly as before. The only required modification is the addition of one dimension to the vector of B-spline nodes. We still use 1D wavelets as we have done previously.

Some prediction of our metamodel and the corresponding error maps are available on the Git repository (see section 8, go to Cas_jouet_Campbell/output-GIF/maps or maps2 and watch the hauteur_eau_reconstruite or hauteur_eau_reconstruite). Overall, we obtained mixed performance. The B-spline–based method appears to be really less effective than the others, although we did not have sufficient time to further investigate the reasons for this behavior. For the two others, as you can see on the two GIFs available, the prediction of our metamodel is a really smooth map. In most cases the prediction is quite accurate with the same range of error (10%) than previously but we identified two scenarios where the prediction is quite far from reality :

- When the input is far from the training set. This scenario is normal as our model never trained on this region (even if we try to cover the space at best with the maximin design)
- When the input lead to discontinuity through time. Even if the Campbell function is really smooth, if the input vary a lot between two time steps, the Campbell function will vary a lot leading to discontinuity through time.

Hopefully we have identified the scenarios where our model is not accurate. It will not be a too serious problem for the real-case study as in real life, no such discontinuity will be observable and we will design our experiments the most optimized way to avoid new inputs two far from a training

one. We will see if we are right to pass directly to the real case study without changing anything.

6.5. Conclusion on the toy model

Working with the toy model allowed us to develop robust tools that are easily transferable to other settings. We took the time to test various configurations, which helped us gain a deeper understanding of the key concepts underlying the effectiveness of this technology.

Now that the results are satisfactory, we can confidently apply our tools to a real-world case.

7. Real-case study: the bay of Saint-Malo

We now apply our methodology to a real-world case study: the Bay of Saint-Malo. The TOLOSA simulator relies on numerical solvers to produce flood forecasts. We benefited from the support of Mr. Bastien Lombard, who provided the simulation data used in this study. The study area is shown in Figure 10.

Figure 10: Study area of the Bay of Saint-Malo, as seen from above.

The simulations were performed using two input parameters: the mean sea level elevation (MSL), ranging from 0 to 1 m, and the significant wave height (Hs), ranging from 1 to 3 m. A total of 55 simulations were conducted, requiring approximately 14 hours of computation.

The simulator outputs are defined on a non-structured mesh, leading to non-structured data. When required, these outputs can be post-processed and projected onto a regular grid (see the provided Python script on Git).

We have access to the following information for each terrestrial cell of the mesh:

- the global cell identifier;
- the (x, y) coordinates of the cell barycenter;
- the bathymetry of the cell;
- the surface area of the cell.

For visualization purposes, the file `gmsh_mapping.vtk` is provided and allows the inspection of the non-structured data. Figure 11 illustrates this representation.

Finally, for each simulation directory, the binary file `scattered_map_final.bin` stores, for each terrestrial cell of the mesh:

- the global cell identifier;
- the maximum water height reached over the entire simulation, denoted $H_{\max,T}$;
- the mean water height over the entire simulation, denoted $H_{\text{mean},T}$.

7.1. The classical approach: scalar to scalar scenario

We have a total of 55 simulations. We chose to retain 50 simulations for training and 5 for testing. Unfortunately, we did not have the opportunity to discuss the design of experiments prior to running the simulations; as a result, the design is not optimal, as illustrated in Figure 12. Nevertheless, the available data remain sufficient to carry out the analysis.

The mesh now contains 488290 cells, which means we are working in $\mathbb{R}^{488290}$. We can directly reuse the methodology developed on the Campbell toy model to reduce dimensionality and perform Gaussian process predictions. We do not use the B-spline-based method, as the choice of the knot vector is unclear. Instead, we opted for the wavelet-based approach.

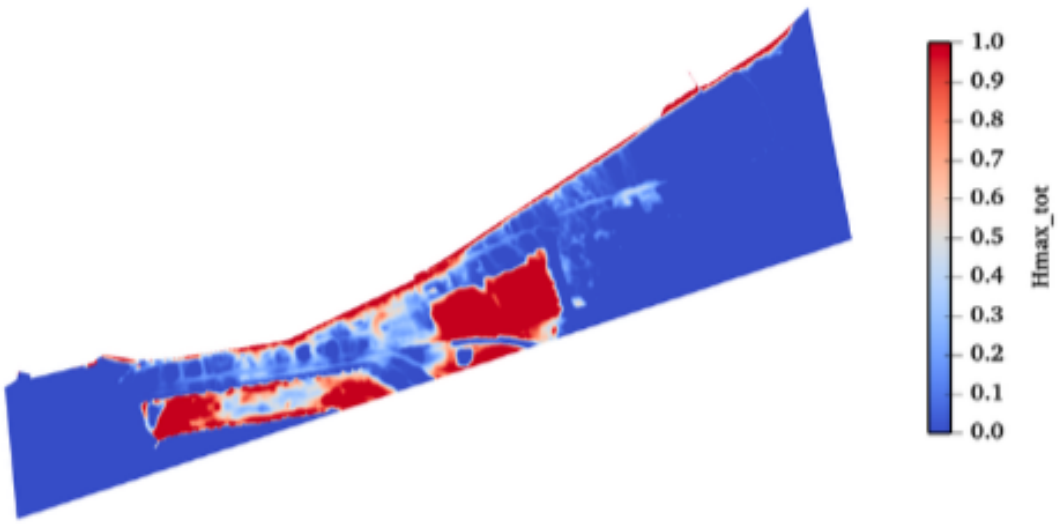

Figure 11: Visualization of output given by the Tolosa simulator.

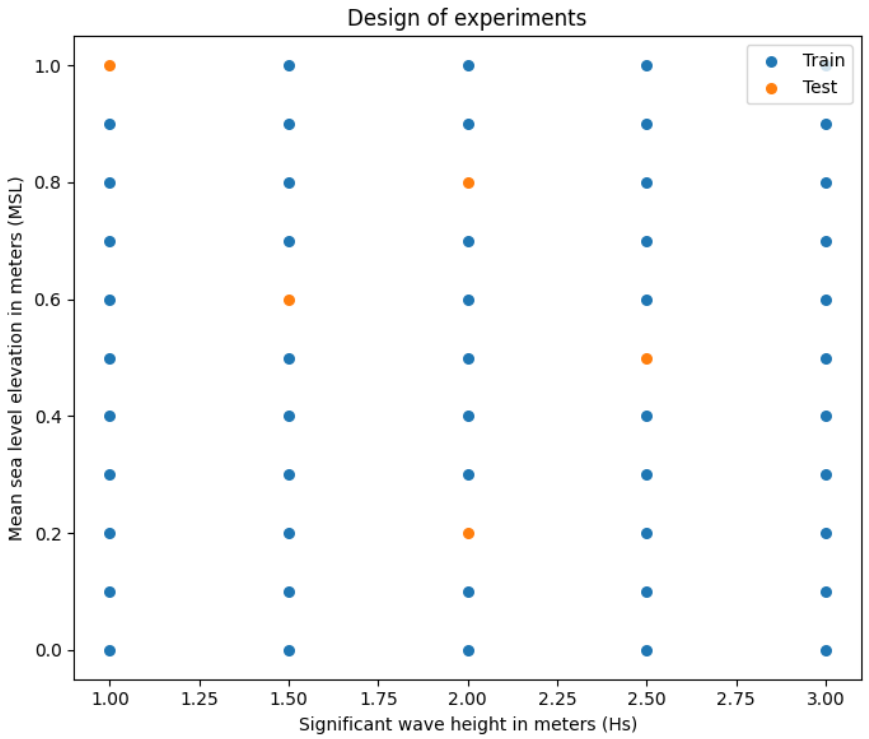

Figure 12: Design of experiments for the Saint-Malo case study, showing the distribution of training and test simulations.

We again retained 5 principal components and trained the metamodels. To test their effectiveness, we predicted outputs for both training and test datasets. Since the model must interpolate the training data, the training error is expected to be very low. We used the root mean squared error (RMSE) (6.1) as a performance metric; the results are shown in Table 4.

We obtain very low errors overall, with the training error (=estimation of the reconstruction error) close but lower than the test error, which is consistent with the model's interpolation behavior. Another time, we conclude that the GPR is performing very well and add only a small amout of error to the reconstruction one. An example of reconstruction for a test input is shown in Figures 13

Dataset	RMSE
Train	0.054
Test	0.084

Table 4: Mean squared error for training (reconstruction error) and test datasets using the wavelet-based approach.

to 15, displaying respectively the true output, the reconstructed output, and the relative error. Additional reconstructions are available in the project's Git repository.

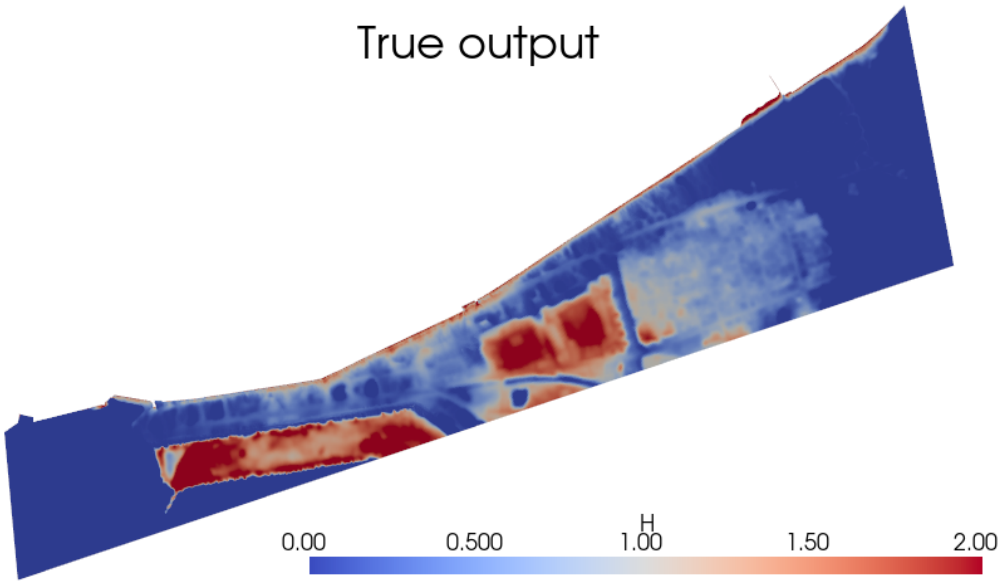

Figure 13: True output for a test input.

The results are highly satisfactory. Some regions in the error map are less reliable due to division by values close to zero when computing relative errors. Overall, the predictions are robust and can be further improved with a better design of experiments.

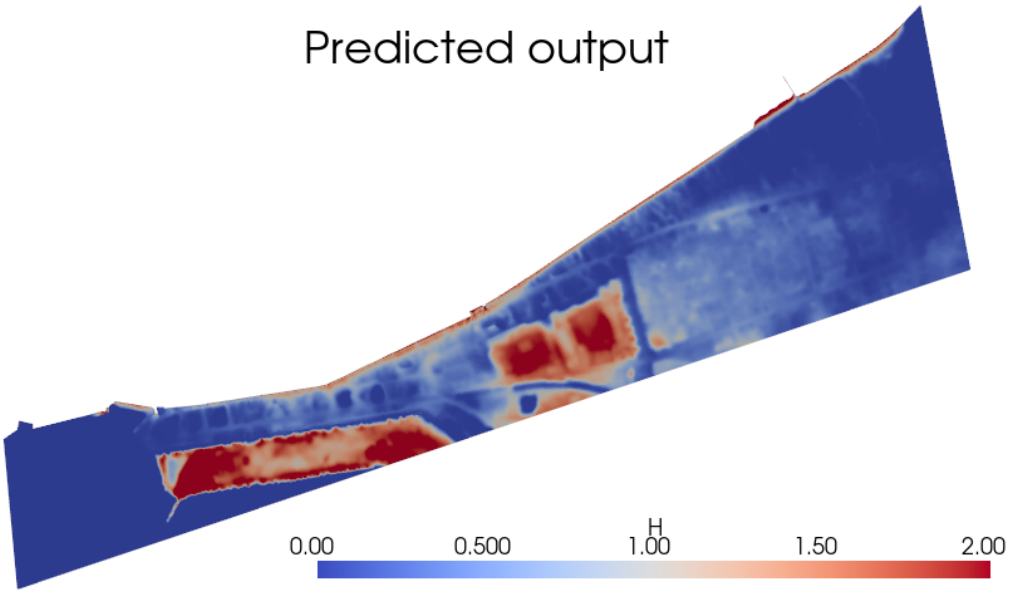

Figure 14: Reconstructed output for the same test input.

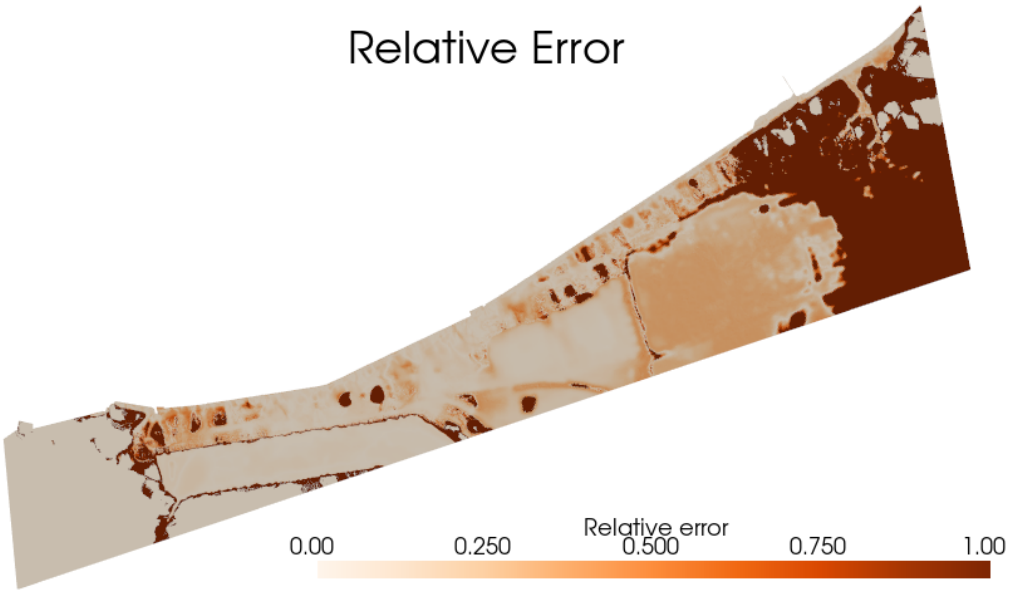

Figure 15: Relative reconstruction error for the same test input.

7.2. Adding temporality

At the end of the study, we explored a case where both the input and the output depend on time. We added a time series containing information on the tide (its exact meaning is unclear to us).

To make the two other input variables compatible, we transformed them into constant functions. Figure 16 illustrates the resulting inputs.

This time, 18 simulations were conducted, requiring approximately 5 hours of computation. We retained 15 for training and 3 for testing. Figure 17 shows the design of experiments for MSL and Hs before converting them into constant functions. The tide information is identical for all inputs.

At this stage, all necessary components are available to apply our methodology. To visualize the results, we generated animated GIFs, which are available here. We can again compute the reconstruction error and the error on the test set with (6.1) (see Table 5).

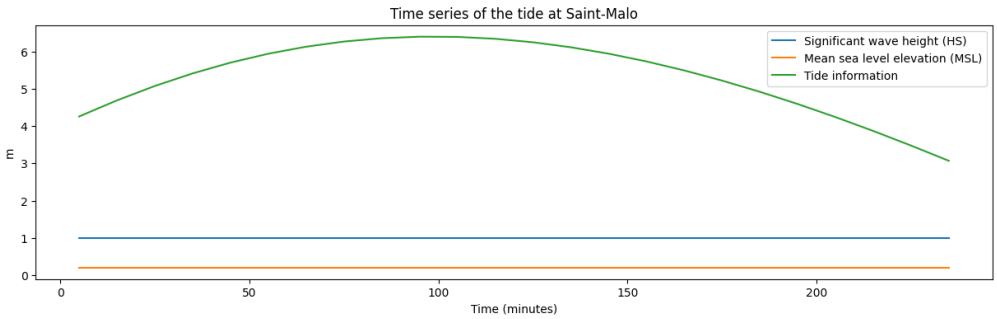

Figure 16: Functional input representation for the Saint-Malo case study, including tide time series and constant functions for MSL and Hs.

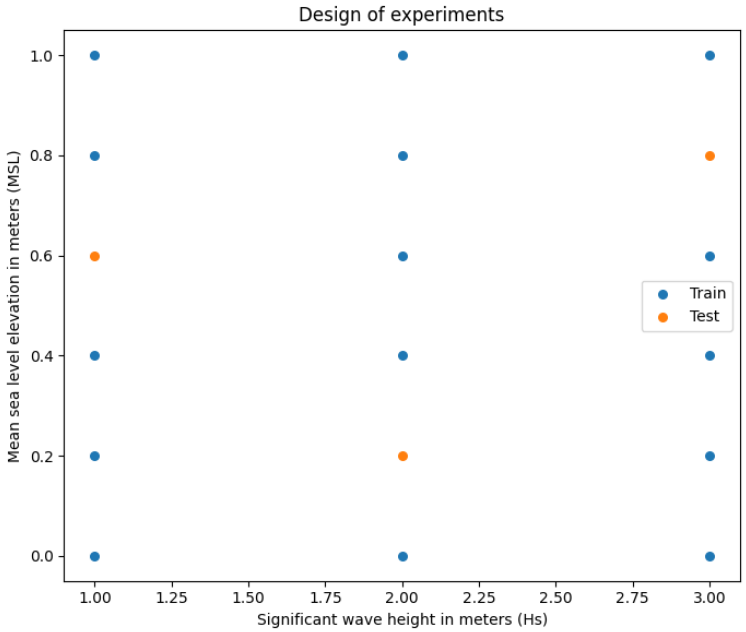

Figure 17: Design of experiments for functional inputs in the Saint-Malo case study, showing MSL and Hs values prior to transformation into constant functions.

Dataset	RMSE
Train	0.04
Test	0.072

Table 5: Mean squared error for training (reconstruction error) and test datasets using the wavelet-based approach.

We can inspect predictions for a small subset of cells. For example, Figure 18 shows the predicted versus true values over time for 5 randomly selected cells. The results are very satisfactory, as the model appears to capture the true dynamics in a consistent manner. For reference, the predictions are generated almost instantaneously using our code, which was one of the main goals of the project.

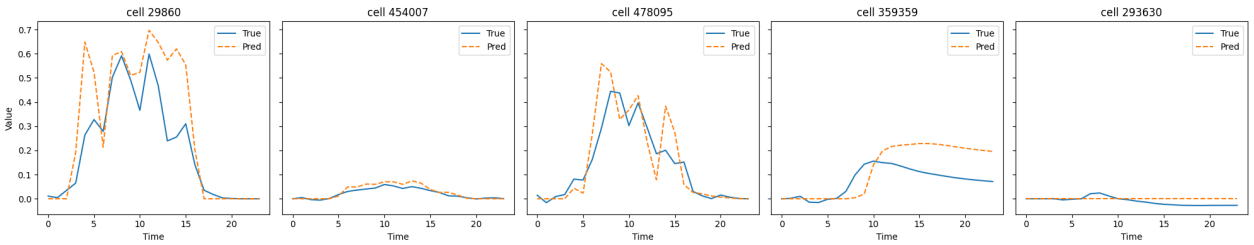

Figure 18: Predicted versus true water heights over time for 5 randomly selected cells in the Saint-Malo case study.

7.3. Conclusion on the real-world application

The real-world application on the Bay of Saint-Malo allowed us to validate the methodology on complex, high-dimensional, temporally dependent, real life data. Despite the limited number of simulations and the non-optimal design of experiments, the wavelet-based Gaussian process meta-model provided encouraging accurate predictions. This is quite impressive as the application on the toy model for the full functionnal framework was not that performant. The thing is the natural behavior of coastal flooding is much more smmother/continuous both in time and space than our toy function. This difference explain why the Gaussian processes are performing better. The approach can be further improved with additional simulations, better experimental designs, and inclusion of more refined temporal and spatial features.

8. Conclusion

To overcome the computational limitations of the TOLOSA physical model, we developed an approach that relies on coupling Functional Principal Component Analysis (FPCA) with Gaussian Process regression. By projecting high-dimensional spatial outputs onto Wavelet or B-spline bases and using a specific kernel adapted for functional inputs (such as tide and weather conditions), we successfully built a metamodel capable of real-time predictions at a city scale.

The methodology was first validated on the Campbell toy function and then applied to the real-world case of the Bay of Saint-Malo. Despite a limited training dataset of 55 simulations and a sub-optimal design of experiments, the Wavelet-based metamodel demonstrated encouraging performance. It achieved low relative errors and satisfactory accuracy, proving the technical feasibility of replacing the physical model with a lightweight statistical model for operational coastal flood forecasting.

However, several avenues for improvement have been identified to possibly upgrade the model. First, the current handling of spatio-temporal outputs via naive flattening could be enhanced by exploring new methods for spatio-temporal decomposition to better capture separate temporal and spatial dynamics as well as spatio-temporal dynamics. Second, while Gaussian Processes inherently provide uncertainty quantification, our implementation focuses on the mean prediction. Exploiting the prediction of variance would be a major asset for risk assessment by simulating multiple possible scenarios. Finally, the design of experiments was constrained in this study. Using space-filling designs or adaptive sampling strategies (EGO algorithm) would enhance the model accuracy.

GitHub repository.

The implementation in Python of the work presented in this paper is available on GitHub. All the informations about this repository are in the README file.

AI Declaration.

This article was written with some use of Gemini 3 and ChatGPT for sentence style refinement and for debugging purposes.

Acknowledgments.

We would like to acknowledge Mrs.NOBLE and ROUSTANT for their support and pedagogy during the whole project. We also want to thank Bastien LOMBARD for the data he provided on the real-case study of Saint-Malo.

REFERENCES

[1] J. Betancourt, F. Bachoca, T. Kleina, D. Idier, R. Pedrerosc, and J. Rohmer, *Gaussian process metamodeling of functional-input code for coastal flood hazard assessment*, Reliability Engineering and System Safety, (2019).

[2] M. de la transition écologique de la biodiversité et des négociations internationales sur le climat et la nature, *Chiffres clés du littoral*, 2025, https://observatoires-littoral.developpement-durable.gouv.fr/chiffres-cles-du-littoral-a17.html. Accessed: 2026-01-10.

[3] R. de Vandeuil, *Loi littoral : alerte sur les côtes!*, L'Express.

[4] M. France, *Vigilance météo france*, 2025, https://vigilance.meteofrance.fr/fr. Accessed: 2026-01-08.

[5] N. Lair, *Ciaran : les tempêtes sont-elles plus fréquentes et plus intenses qu'auparavant en france ?*, France Inter, (2023).

[6] Meteo-France, *L'évolution constatée du climat*, 2025, https://meteofrance.com/climathd. Accessed: 2026-01-10.

[7] T. Muehlenstaedt, J. Fruth, and O. Roustant, *Computer experiments with functional inputs and scalar outputs by a norm-based approach*, Statistics and Computing, 27 (2016), pp. 1083–1097.

[8] T. V.-V. E. Perrin, *Méta-modélisation et analyse de sensibilité pour les modèles avec sortie spatiale. Application aux modèles de submersion marine*, PhD thesis, Université de Lyon, 2021.

[9] O. Roustant, *Metamodeling with Gaussian processes*. Lecture, Sept. 2024, https://hal.science/hal-04509167.